Data & Knowledge Engineering 117 (2018) 98–113

Contents lists available at ScienceDirect

Data & Knowledge Engineering

journal homepage: www.elsevier.com/locate/datak

Improved suffix blocking for record linkage and entity resolution

Amin Allam[a,*], Spiros Skiadopoulos[b], Panos Kalnis[a]

[a] King Abdullah University of Science and Technology, Thuwal 23955-6900, Saudi Arabia
[b] University of the Peloponnese, Tripoli 22100, Greece

ARTICLE INFO

Keywords:
Database integration
Data warehouse and repository

ABSTRACT

Record linkage is the problem that identifies the different records that represent the same real-world object. *Entity resolution* is the problem that ensures that a real-world object is represented by a single record. The *incremental* versions of record linkage and entity resolution address the respective problems after the insertion of a new record in the dataset. Record linkage, entity resolution and their incremental versions are of paramount importance and arise in several contexts such as data warehouses, heterogeneous databases and data analysis. *Blocking* techniques are usually utilized to address these problems in order to avoid comparing all record pairs. *Suffix blocking* is one of the most efficient and accurate blocking techniques. In this paper, we consider the non-incremental variation of record linkage and present a method that is more than five times faster and achieves similar accuracy to the current state-of-the-art suffix-based blocking method. Then, we consider the incremental variation of record linkage and propose a novel incremental suffix-based blocking mechanism that outperforms existing incremental blocking methods in terms of blocking accuracy and efficiency. Finally, we consider incremental entity resolution and present two novel techniques based on suffix blocking that are able to handle the tested dataset in a few seconds (while a current state-of-the-art technique requires more than eight hours). Our second technique proposes a novel method that keeps a history of the deleted records and the merging process. Thus, we are able to discover alternative matches for the inserted record that are not possible for existing methods and improve the accuracy of the algorithm. We have implemented and extensively experimentally evaluated all our methods. We offer two implementations of our proposals. The first one is memory-based and offers the best efficiency while the second one is disk-based and scales seamlessly to very large datasets.

1. Introduction

The problem of identifying and consolidating different manifestations of the same real-world object is of paramount importance. This problem arises in the context of data warehouses [1], heterogeneous databases [2,3] and data analysis and mining [4] and pertains to data cleansing. *Duplicate-free* datasets are also needed in a plethora of modern applications including vital records [5], health care [6], and insurance and crime investigation [3]. In this work, we consider two variations of the aforementioned problem. The first one, termed *record linkage* [5,7], identifies the different records that represent the same real-world object. The second one, termed *entity resolution*, ensures that a real-world object is represented by a single record by also consolidating the identified (by record linkage) records [8]. Entity resolution is an inherently recursive process; the resulting record from a consolidation may be different from each of the original individual records that constituted it, so it should also be checked whether it identifies the same

* Corresponding author.
E-mail addresses: amin.allam@kaust.edu.sa (A. Allam), spiros@uop.gr (S. Skiadopoulos), panos.kalnis@kaust.edu.sa (P. Kalnis).

https://doi.org/10.1016/j.datak.2018.07.005
Received 28 June 2016; Received in revised form 23 March 2018; Accepted 22 July 2018
Available online 30 July 2018
0169-023X/ © 2018 Elsevier B.V. All rights reserved.

real-world object with an existing record in the dataset (in which case entity resolution is recursively executed) or not (in which case the process terminates).

Recent advances in modern applications require incremental solutions for the record linkage and entity resolution problems [1,3,9]. Take for example the data warehouse applications. Record linkage (or duplicate detection) and entity resolution are common operations in data warehouses, for the purpose of data cleansing. These operations are usually done in batched mode, during the Extraction-Transformation-Loading process. The downside of performing such operations in batched mode is that the warehouse is not consistently up-to-date with respect to the latest updates. To address this issue, modern approaches introduce *active* (or *real-time*) data warehouses that are consistently up-to-date with their transactional counterparts [1]. The key idea lies on the immediate propagation of every modification that happens in the transactional system to the data warehouse. This calls for *incremental* variations of the applied record linkage and entity resolution solutions.

In more details, given a dataset D, *incremental record linkage* considers a new record r and identifies the records in D that represent the same real-world object with r [10]. *Incremental entity resolution* ensures that after the addition of the new record r, the real-world object that corresponds to r is represented by a single record in D [11]. In other words, if D is duplicate-free, incremental entity resolution maintains this property after the insertion of r. Similarly to entity resolution, the incremental variation is also a recursive process; a consolidated record should be recursively checked for matches.

Incremental record linkage and incremental entity resolution can be solved more efficiently than their non-incremental counterparts. For instance, to address incremental record linkage, we only need to compare the inserted record r with the records of the dataset D. Note that a non-incremental algorithm will also compare all pairs of records in D which clearly is not required by the incremental case. Therefore, applying the non-incremental methods to the incremental problems is not efficient and new algorithms that directly address the incremental variations should be devised.

An *ordinary* method to identify the records in a dataset D that represent the same real-world object is to compare all pairs of records in D. This quadratic cost algorithm is prohibitive for large datasets [12]. To improve efficiency, approximate techniques were introduced. Such methods apply a preprocessing heuristic, called *blocking*, that efficiently excludes records that are not believed to be similar [12]; the remaining records are compared using the ordinary method. The reduction achieved by blocking may result in a significant efficiency improvement, but, it may also reduce accuracy since the applied heuristic may falsely exclude valid linked records. Several blocking-based techniques were introduced for the non-incremental cases and include Multi-pass blocking [13,14], Q-gram based indexing [9,15,16], Canopy clustering [17], Suffix blocking [18,19], Mapping [4], MFIBlocks [20], and Meta-blocking [2,21]. Unfortunately, these methods cannot directly be used for the incremental case. Some like Multi-pass blocking [13,14] and Q-gram based indexing [9,15,16] need some modifications [22], while others like Canopy clustering [17], Suffix blocking [18,19], Mapping [4], MFIBlocks [20], and Meta-blocking [2,21] need a complete redesign.

One of the most prominent blocking methods is based on suffixes [20,23]. In more details, suffix-based blocking methods construct a structure combining the suffixes of all records in the dataset D [18,19]. This structure is used to identify the pairs of records that are believed to manifest the same real-world entity. Suffix-based blocking is both efficient and accurate and does not require any domain specific knowledge [18]. Additionally, it can be tuned by only two easy-to-set and intuitive parameters [19]. For the above reasons, in this paper, we adopt the idea of using a suffix-based preprocessing heuristic and apply it in order to solve the problems of record linkage and entity resolution.

We start by considering the non-incremental variation of record linkage and present a suffix-based blocking method for this task. In comparison with the current state-of-the-art suffix-based blocking method, our proposal is more than five times faster and achieves similar accuracy. Then, we consider the incremental variation of record linkage. To address this problem, we propose a novel incremental suffix-based blocking mechanism. Our proposal outperforms existing incremental methods. Specifically, our method offers better blocking accuracy since, for the same number of excluded record links, it has more than 3% and up to 50% increase in pairs completeness. Also, our method is more efficient since, for the same pairs completeness accuracy, it is more than two and up to eight times faster.

Finally, we consider incremental entity resolution. Existing methods are quite inefficient. For instance, a state-of-the-art method requires more than eight hours to handle a dataset of 20, 000 records [11]. To improve the efficiency of incremental entity resolution, we present two novel methods based on suffix blocking that are able to handle the above mentioned dataset of 20, 000 records in a few seconds. Our second method, proposes a novel method that keeps a history of the deleted records and the merging process. This idea gives us the opportunity to discover alternative matches for the inserted record that are not possible for existing methods that remove the merged records from the dataset and improves the accuracy of the algorithm.

We have implemented and extensively experimentally evaluated all our methods. We offer two implementations of our proposals. The first one is memory-based and offers the best efficiency. The second one is disk-based and scales seamlessly to very large datasets that cannot fit in main memory. Surprisingly, with very few exceptions [19], current state-of-the-art methods do not offer disk-based implementations.

The rest of this paper is organized as follows: Section 2 provides preliminaries for record linkage. Section 3 introduces an improved blocking method for record linkage. In Section 4, we present a novel blocking method for incremental record linkage. Section 5 introduces two novel blocking methods for incremental entity resolution. In Section 6, we present our experimental analysis. Section 7 surveys related work. Finally, Section 8 concludes the paper.

2. Preliminaries

Record linkage on a dataset D can be trivially done by a comparison method that extensively compares all record pairs of the

ID	Name	Address
r_1	jim mat	ny usa
r_2	vic mat	ny usa
r_3	mat jim	ny usa
r_4	jim mat	wa usa
r_5	jim vic	ny usa

(a) A dataset D

	Record suffix	Record ID	LCP
1	jim mat ny usa	r_1	0
2	jim mat wa usa	r_4	2
3	jim ny usa	r_3	1
4	jim vic ny usa	r_5	1
5	mat jim ny usa	r_3	0
6	mat ny usa	r_1	1
7	mat ny usa	r_2	3
8	mat wa usa	r_4	1
9	ny usa	r_1	0
10	ny usa	r_5	2
11	ny usa	r_3	2
12	ny usa	r_2	2
13	usa	r_1	0
14	usa	r_4	1
15	usa	r_5	1
16	usa	r_3	1
17	usa	r_2	1
18	vic mat ny usa	r_2	0
19	vic ny usa	r_5	1
20	wa usa	r_4	0

(b) The sorted suffixes *SortSuf* and array *LCP* corresponding to D

Fig. 1. Example of a dataset and suffix-based blocking structures.

dataset D. This method requires $\mathcal{O}(|D|^2)$ comparisons. Thus, if the dataset D is large this method is prohibitively expensive; even for datasets of hundred thousand records, it might require months to complete.

To address this efficiency issue, *blocking methods* for record linkage were introduced [4,13,15,17–19]. These methods are performed in two stages, namely a blocking and a comparison stage. The crux of such methods, is the *blocking stage*, that applies a fast heuristic method to exclude pairs of records that are not believed to be linked. The remaining pairs are forwarded to the *comparison stage*, that just verifies (typically using a similarity function) whether the corresponding records are linked or not. The reduction achieved by the blocking stage may result in a significant efficiency improvement, but, it may also reduce accuracy since the heuristic may falsely exclude valid linked pairs. To address this efficiency issue, *blocking methods* for record linkage were introduced [4,13,15,17–19]. These methods are performed in two stages, namely a blocking and a comparison stage. The crux of such methods, is the *blocking stage*, that applies a fast heuristic method to exclude pairs of records that are not believed to be linked. The remaining pairs are forwarded to the *comparison stage*, that just verifies (typically using a similarity function) whether the corresponding records are linked or not. The reduction achieved by the blocking stage may result in a significant efficiency improvement, but, it may also reduce accuracy since the heuristic may falsely exclude valid linked pairs.

One of the most prominent blocking methods is based on suffixes [20,23]. In more detail, *suffix-based blocking* methods construct and sort the suffixes of all records in the dataset [18,19]. Consider for example the dataset D of Fig. 1a; the sorted suffixes of the records of the dataset, denoted by *SortSuf* (D), are illustrated in Fig. 1b. *SortSuf* (D) is typically coupled with a helper array named *LCP*. *LCP*[i] is the number of subsequent starting common words between the suffixes *SortSuf* [i] and *SortSuf* [$i-1$] (by definition *LCP* [1] = 0). In Fig. 1b, we have *LCP* [2] = 2 since the suffixes *SortSuf* [1] and *SortSuf* [2] share a subsequence of 2 starting common words jim mat. Similarly, *LCP* [19] = 1 since the suffixes *SortSuf* [18] and *SortSuf* [19] share only the starting common word vic. Recent suffix-based blocking methods, identify sets of records that correspond to blocks of suffixes in *SortSuf* (D) and use these sets to create the pairs that are forwarded to the comparison stage. The blocks of suffixes are identified based on two parameters [18]. The first one, denoted by min_{LCP}, determines the minimum number of subsequent starting common words between the suffixes of a block. Sets of records that correspond to blocks of suffixes in *SortSuf* (D) having min_{LCP} or more subsequent starting common words are saved for further processing; blocks having less are discarded. The second parameter, denoted by max_{APP}, sets the maximum allowed number of appearances of the common subsequences in the entire dataset. Sets of records that correspond to blocks of suffixes in *SortSuf* (D) that their subsequent common words appear max_{APP} or less times in the entire dataset are saved for further processing; blocks with more are discarded. For each such set of records B, suffix-based blocking methods forward to the comparison stage the pairs of records that can be formed by the records of B. For example, consider Fig. 1. Setting $max_{APP} = 3$ and $min_{LCP} = 2$ renders a suffix-based blocking method to result in two blocks of suffixes of Fig. 1b: the block [1,2] that shares jim mat and the block [6,7] that shares mat ny; these blocks correspond to records $\{r_1, r_4\}$ and $\{r_1, r_2\}$ respectively (see also Fig. 1b).

3. Record linkage

In this paper, we consider suffix-based blocking methods for record linkage. In fact, we focus on the blocking stage of such methods and particularly on the identification of the sets of records that correspond to suffix blocks that satisfy min_{LCP} and max_{APP} constraints and, thus, deserve further processing. This is the most expensive part of these methods and, thus, the one that deserves to be optimized. After these sets of records are identified, producing the pairs to be compared and performing the comparison stage is trivial.

The SAB suffix-blocking method for record linkage [18], to enforce min_{LCP} and max_{APP} constraints, considers every suffix with size

greater or equal to min_{LCP}. For each such suffix s, SAB considers the first min_{LCP} words of s and locates the largest block B in the array of sorted suffixes that start with these words. If the size of block B is less than max_{APP}, the set of records that correspond to B is saved for further processing and SAB considers the following suffix. Otherwise, SAB progressively increases the working number of words (to $min_{LCP} + 1$, $min_{LCP} + 2$, etc.) and proceeds similarly. To present a concrete example, consider Fig. 1b and parameters $min_{LCP} = 1$ and $max_{APP} = 3$. Since $min_{LCP} = 1$, SAB considers all suffixes of D (illustrated in Fig. 1b). Thus, SAB first considers suffix s = jim mat ny usa and searches for the block B starting with jim which is formed by the first 4 rows of Fig. 1b. Since the size of B exceeds max_{APP}, SAB searches for a new block starting with jim mat which is formed by the first 2 rows of Fig. 1b. Now, since the block size is smaller than $max_{APP} = 3$, SAB saves the records that correspond to this block (that is, r_1 and r_4) for further processing and moves to the next suffix.

For each considered suffix s, SAB [18] searches the array of sorted suffixes $\mathcal{O}(|s|)$ times. In this section, we will develop a method that, for each suffix, performs *only one* search. Thus, our proposal produces the same result as SAB (and of course have the same accuracy) but it is much more efficient. Our approach, illustrated by Algorithm SUFFBLOCK, uses a sliding window of increasing size. Using this window, we are able to apply both min_{LCP} and max_{APP} parameters simultaneously without missing eligible candidates. The process is achieved very efficiently without unnecessary checks. Algorithm SUFFBLOCK uses a working and a saved window denoted by w and w_S respectively. The length of the longest common subsequence of all suffixes in the working window w is maintained in the cur_{LCP} variable. The working window w is progressively extended provided that the min_{LCP} and max_{APP} constraints are not violated. Moreover, w is saved to the stored window w_S if during the extension cur_{LCP} is decreased. The output of the algorithm *Saved* consists of a set of sets; each set contains candidate records that deserve further processing.

Algorithm: SUFFBLOCK

Input: The sorted suffixes *SortSuf* and array LCP of a dataset D
Output: The set *Saved* of sets of records that require further processing
Parameters: max_{APP} and min_{LCP}
Local variables: i, cur_{LCP}, w, w_S

```
1  Let Saved = ∅, i = 1, LCP[ |LCP| + 1 ] = 0
2  while i ≤ |LCP| do
3      Let w = [w⁻, w⁺] be the working window; initially w⁻ = w⁺ = i
4      Let w_S = [w_S⁻, w_S⁺] be the saved window; initially w_S = w
5      cur_LCP = LCP[i + 1]
6      while min_LCP ≤ cur_LCP, |w| ≤ max_APP and LCP[w⁻] < cur_LCP do
7          if LCP[w⁺ + 1] < cur_LCP then
8              w_S = w
9              cur_LCP = LCP[w⁺ + 1]
10         w⁺ = w⁺ + 1            // Extend window w
11     if |w_S| > 1 then
12         Add to Saved the set that contains all records corresponding to the
           window w_S
13     i = w_S⁺ + 1
14 return Saved
```

In more details, Algorithm SUFFBLOCK proceeds as follows. Line 1 initializes variables. Then, it performs a while loop (Lines 2–13) that scans array *LCP* of the dataset *D*. Starting at the current position *i*, SUFFBLOCK initializes the working windows w and the saved window w_S and sets $cur_{LCP} = LCP[i + 1]$ (Lines 3–4). Following, as long as the min_{LCP} and max_{APP} constraints are not violated (that is, $min_{LCP} \leq cur_{LCP}$, $|w| \leq max_{APP}$ and $LCP[w^-] < cur_{LCP}$), the algorithm extends w (Line 10). If during this extension, the value of *LCP* array is decreased (Line 7) the current window w is saved to w_S (Line 8). Then, Line 12 adds to the output *Saved* the set that contains all records corresponding to the saved window w_S (provided that there are more than one). Finally, SUFFBLOCK continues just after the end of the saved window (Line 13) and repeats Lines 2–13.

Example 1. *Let us execute Algorithm* SUFFBLOCK *with input the sorted suffixes SortSuf and array LCP (*Fig. 1b*) that correspond to the dataset D of* Fig. 1a *and parameters* $max_{APP} = 3$ *and* $min_{LCP} = 2$. *After initializations, the algorithm enters the outer* **while** *loop (Line 2) and sets* $w = w_S = [1,1]$ *and* $cur_{LCP} = 2$. *At the first iteration of the inner* **while** *loop (Line 6), w is extended to* $w = [1,2]$. *At the second iteration, we have* $w_S = [1,2]$, $cur_{LCP} = 1$ *and* $w = [1,3]$. *Following,* SUFFBLOCK *at Line 12 adds to the output the set containing records* r_1 *and* r_4 *corresponding to* $w_S = [1,2]$ *for further processing, sets* $i = 3$ *and moves to Line 2. In a similar manner,* SUFFBLOCK *processes the remaining records of D, and also adds to the output the set containing records* r_1 *and* r_2 *corresponding to* $w_S = [6,7]$. *The final result is illustrated in* Fig. 2a. *Note that* SUFFBLOCK, *considers for further processing records* r_1 *and* r_4 *sharing* jim mat *while it does not consider records* r_1 *and* r_2 *that share the much frequent* ny usa. *Setting* $min_{LCP} = 1$ *renders* SUFFBLOCK *to return an additional set that contains records* r_2 *and* r_5 *corresponding to* $w_S = [18,19]$. *The final result is illustrated in* Fig. 2b.

Setting $min_{LCP} = 1$ renders Algorithm SUFFBLOCK to consider two records for similarity test if they have a common sequence of any

r_1	jim mat	ny usa
r_4	jim mat	wa usa
r_1	jim mat	ny usa
r_2	vic mat	ny usa

(a) $max_{APP} = 3$ and $min_{LCP} = 2$

r_1	jim mat	ny usa
r_4	jim mat	wa usa
r_1	jim mat	ny usa
r_2	vic mat	ny usa
r_2	vic mat	ny usa
r_5	jim vic	ny usa

(b) $max_{APP} = 3$ and $min_{LCP} = 1$

Fig. 2. Output of SUFFBLOCK with input dataset D of Fig. 1a on different parameter values (Example 1).

size provided that it is not very common in the dataset (see also Example 1).

Theorem 1. *Algorithm* SUFFBLOCK *outputs all subsets of a dataset D having a common sequence of at least* min_{LCP} *words unless this common sequence appears more than* max_{APP} *times in D.*

Proof. Let us assume that there is a set of m records having a common sequence *seq* of words of size at least min_{LCP}. There exist a suffix for each of these m records starting with the common sequence *seq*. These suffixes will be adjacent in the sorted record suffixes list. If the number of these suffixes is at most max_{APP}, they will occur in the same sliding window and their associated records will be returned by Algorithm SUFFBLOCK else they are discarded.□

Proposition 1. *The time complexity of Algorithm* SUFFBLOCK *is* $\mathcal{O}(|D|\cdot|r|\cdot max_{APP})$ *where* $|r|$ *is the maximum record size.* SUFFBLOCK *can be performed with a single scan of SortSuf* (D). *The overall time complexity of the blocking process, that is, constructing the SortSuf* (D) *array and executing* SUFFBLOCK, *is* $\mathcal{O}(|D|\cdot|r|\cdot\log(|D|\cdot|r|) + |D|\cdot|r|\cdot max_{APP})$

Proof. The number of iterations of the outer **while** loop is $\mathcal{O}(|D|\cdot|r|)$. The number of iterations of the inner **while** loop is $\mathcal{O}(max_{APP})$ (in each loop w is extended by one and it is always $|w| \leq max_{APP}$). In total, SUFFBLOCK can be performed in $\mathcal{O}(|D|\cdot|r|\cdot max_{APP})$ time.

SUFFBLOCK extends the working window w by reading sequentially the sorted suffixes of *SortSuf* (D) (Line 10). SUFFBLOCK only needs to go back when the saved window is smaller than the working window (Line 13). Such cases can be handled with caching. In our case, we only need to cache $max_{APP}/P + 1$ pages, where P is the disk page size.

Since the time complexity of constructing the *SortSuf* (D) array is $\mathcal{O}(|D|\cdot|r|\cdot\log(|D|\cdot|r|))$, the overall time complexity of the blocking process is $\mathcal{O}(|D|\cdot|r|\cdot\log(|D|\cdot|r|) + |D|\cdot|r|\cdot max_{APP})$.□

Algorithm SUFFBLOCK outperforms SAB mainly for the following two reasons. First, SUFFBLOCK incorporates a dynamic sliding window that requires only sequential input scans. Thus, it is very cache efficient. Contrary SAB conducts at least two binary searches on the *SortSuf* array for each record suffix, to locate the start and end of the block containing similar suffixes. Such searches suffer from poor locality and are significantly more expensive than sequential scans. Additionally, SUFFBLOCK avoids iterating over all the prefixes of a certain suffix as SAB does. Overall, as the experimental evaluation of Section 6 indicates SUFFBLOCK outperforms SAB by more than an order of magnitude.

Algorithm SUFFBLOCK works exclusively in main memory, that is, all data, data structures and processing is performed in main memory. To remove the bounds set by the main memory size and improve SUFFBLOCK scalability, we may also incorporate external storage. To this end, we use the disk-based algorithm of [24] to construct the sorted suffixes *SortSuf* and array *LCP*. The *LCP* array elements are interleaved with their corresponding Record IDs (Fig. 1b). In more detail, each *LCP* value (4th column of Fig. 1b) is stored next to its Record ID (3rd column of Fig. 1b) on disk. For example, the first row of Fig. 1b is stored as [0, r_1] on disk. Proposition 1 also applies to the disk-based case, thus, SUFFBLOCK can be performed with a single scan. To execute the disk-based version of SUFFBLOCK, we need to store in main memory only the working window w, all other structures may reside in external storage.

4. Incremental record linkage

In this section, we propose a blocking method for incremental record linkage. In the spirit of Section 3, we develop a suffix-based blocking method and focus on the identification of the saved sets of records that correspond to suffix blocks of the *SortSuf* array that satisfy min_{LCP} and max_{APP} constraints.

Similarly to the non-incremental case (Section 3), our approach takes advantage of the sorted suffixes array *SortSuf*. To support efficient incremental operators, we employ a StrB-tree [25] index *SB* on *SortSuf*, augmented with *LCP* values [26]. StrB-tree supports efficient searching and incremental updates (both operations can be performed in $\mathcal{O}(\log |D|)$ time); more details appear in Refs. [25,26]. Our method is illustrated in Algorithm INCSUFFBLOCK. To insert a new record r in dataset D, INCSUFFBLOCK iterates over all suffixes of r (Line 2–19). INCSUFFBLOCK inserts suffix s into StrB-tree *SB* and constructs a working window w around the inserted location. Initially, the window contains only the inserted suffix s. Then, w is expanded to both directions (using the bidirectional list of the StrB-tree *SB* connecting the leaf nodes) as much as possible while preserving the min_{LCP} and max_{APP} constraints. The method of constructing the window is conceptually similar to Algorithm SUFFBLOCK. The prominent difference is that the case of INCSUFFBLOCK the working window can be expanded to both directions.

Algorithm: IncSuffBlock

Input: A new record r and the sorted suffixes *SortSuf* of a dataset D indexed by a StrB-tree SB
Output: The set *Saved* of sets of records that require further processing
Parameters: max_{APP} and min_{LCP}
Local variables: i, cur_{LCP}, w, w_S, s

1 Let $Saved = \emptyset$, $LCP[|LCP| + 1] = 0$
2 **for** *each suffix* s *of* r **do**
3 Insert s into *SortSuf* using SB
4 Let i be the location of r_S in *SortSuf*
5 Let $w = [w^-, w^+]$ be the working window; initially $w^- = w^+ = i$
6 Let $w_S = [{w_S}^-, {w_S}^+]$ be the saved window; initially $w_S = w$
7 $cur_{LCP} = max(LCP[i], LCP[i + 1])$
8 **while** $min_{LCP} \leq cur_{LCP}$ *and* $|w| \leq max_{APP}$ **do**
9 **if** $LCP[w^-] > LCP[w^+ + 1]$ **then**
10 **if** $LCP[w^-] < cur_{LCP}$ **then**
11 $w_S = w$
12 $cur_{LCP} = LCP[w^-]$
13 $w^- = w^- - 1$ `// Extend window` w `to the left`
14 **else**
15 **if** $LCP[w^+ + 1] < cur_{LCP}$ **then**
16 $w_S = w$
17 $cur_{LCP} = LCP[w^+ + 1]$
18 $w^+ = w^+ + 1$ `// Extend window` w `to the right`
19 **if** $|w_S| > 1$ **then** Add to *Saved* the set that contains all records corresponding to the window w_S
20 **return** *Saved*

In more details, Algorithm incsuffblock to insert a new record *r* proceeds as follows. Line 1 initializes variables. Then, it performs a while loop (Lines 2–19) iterating over the suffixes of *r*. Every suffix *s* is inserted into *SortSuf* (Line 3) and the working and saved windows are initiated to the inserted location (Lines 5 and 6 respectively). Also, Line 7 sets cur_{LCP} to the maximum of $LCP[i]$ and $LCP[i + 1]$. Following, as long as the min_{LCP} and max_{APP} constraints are not violated (that is, $min_{LCP} \leq cur_{LCP}$ and $|w| \leq max_{APP}$), the algorithm considers two cases:

- if $LCP[w^-] > LCP[w^+ + 1]$, window *w* is extended to the left (Line 13).
- if $LCP[w^-] \leq LCP[w^+ + 1]$, window *w* is extended to the right (Line 18).

In both cases, if during this extension, the value of *LCP* array is decreased (Lines 10 and 15) the current window *w* is saved to w_S (Lines 11 and 16) and cur_{LCP} is updated accordingly (Lines 12 and 17). Then, Line 19 adds to the output *Saved* the set that contains all records corresponding to the saved window w_S (provided that there are more than one) and proceeds to the next suffix.

Example 2. *Let us execute Algorithm* incsuffblock *with input the sorted suffixes SortSuf and array LCP (*Fig. 1b*) that correspond to the dataset D of* Fig. 1a*, the new record* r_6 = jim mat tx usa *and parameters* $max_{APP} = 3$ *and* $min_{LCP} = 2$*. The algorithm iterates over all suffixes of* r_6 *(Lines 2–7). Let us assume that the algorithm first considers suffix* jim mat tx usa. Line 3 of incsuffblock *inserts the suffix into the proper location in SortSuf using the StrB-tree SB (illustrated in* Fig. 3*). Then, the algorithm sets* $w = w_S = [2,2]$ *and* $cur_{LCP} = 2$ *(Lines 5–7). At the first iteration of the* **while** *loop (Lines 8–18), the algorithm extends w to* [2,3] *while, at the second iteration, it extends w to* [1,3]*. At the third iteration, the algorithm sets* $w_S = [1,3]$, $cur_{LCP} = 1$ *and* $w = [1,4]$*. Following,* incsuffblock *at Line 19 returns a set containing records* $\{r_1, r_6, r_4\}$ *corresponding to* $w_S = [1,3]$ *for further processing, and moves to next suffix.*

	Record suffix	Record ID	*LCP*
1	jim mat ny usa	r_1	0
2	jim mat tx usa	r_6	2
3	jim mat wa usa	r_4	2
4	jim ny usa	r_3	1
⋮	⋮	⋮	⋮

Fig. 3. Inserting suffix {jim mat tx usa}.

Theorem 2. *Algorithm* INCSUFFBLOCK *inserts a new record r in a dataset D and outputs all records of the new dataset* $D \cup \{r\}$ *having a common sequence with r of at least* min_{LCP} *words unless this common sequence appears more than* max_{APP} *times in* $D \cup \{r\}$.

Proof. For each suffix s of r, Algorithm INCSUFFBLOCK inserts s in the StrB-tree SB, then it constructs a window that initially contains s and expands it to both directions. If there exist a set of $m \leq max_{APP}$ suffixes (including s) prefixed by a common sequence of words of size at least min_{LCP}, they must occur inside the constructed window and the associated records will be outputted by INCSUFFBLOCK, otherwise they will be discarded. If there exist a record q that shares a common sequence seq with r, there must be a suffix s of r that is prefixed by seq. If seq satisfies the above min_{LCP} and max_{APP} constraints, the expanded window around s must contain a suffix of q which is prefixed by seq as well, hence q will be outputted by INCSUFFBLOCK.□

Proposition 2. *The time complexity of Algorithm* INCSUFFBLOCK *is* $\mathcal{O}(|r|\cdot(\log |D| + max_{APP}))$, *where* $|r|$ *is the size of the inserted record r. Similarly to* SUFFBLOCK, *the time complexity does not depend on the value of* min_{LCP}. *Also,* INCSUFFBLOCK *performs* $\mathcal{O}(|r|\cdot(\log_P |D| + max_{APP}/P))$ *disk accesses (where P is the disk page size) and outputs at most* $|r|\cdot max_{APP}$ *records.*

Proof. The number of the **for** iterations (Lines 2–19) is $\mathcal{O}(|r|)$. In each iteration, the algorithm *(a)* inserts a suffix into the StrB-tree which can be done in $\mathcal{O}(\log |D|)$ time) and *(b)* executes a **while** loop (Lines 8–18) which can be done in $\mathcal{O}(max_{APP})$ time (to verify this, note that for the **while** loop, initially $|w| = 1$, $|w|$ is incremented by one and the loop terminates when $|w| > max_{APP}$). Thus, the total complexity of INCSUFFBLOCK is $\mathcal{O}(|r|\cdot(\log |D| + max_{APP}))$. The disk access bound can be computed similarly. Finally, for each **for** iteration, INCSUFFBLOCK outputs at most max_{APP} records, thus in total, the algorithm outputs at most $|r|\cdot max_{APP}$ records.□

Algorithm INCSUFFBLOCK works exclusively in main memory. Similarly to SUFFBLOCK (Section 3), to remove the bounds set by the main memory size and improve INCSUFFBLOCK scalability, we may also incorporate external storage. To this end, we horizontally divide the StrB-tree SB into two parts. The upper part, stored in main memory, contains the root of SB. Additionally, if there is room in main memory, it might contain some of SB's top levels. The lower part, stored in external storage, contains the remaining levels of SB and definitely the leaf nodes. The number of the top levels of SB stored in main memory is determined by the size of the memory allotted for the record linkage process. Obviously, efficiency is improved when more levels are stored in main memory. To maintain the StrB-tree, we appropriately modify the algorithms of [25,26].

5. Incremental entity resolution

Incremental entity resolution extends the problem of incremental record linkage (that inserts a new record r in a dataset and identifies a group of records in the dataset representing the same real-world entity with r, see also Section 4) by merging the matching records. The process is recursively repeated, i.e., the merged record is, in turn, checked for similar records and the matches are merged. The incremental entity resolution process completes in the iteration that there are no matches for the input record; in such case the input record is added to the dataset and the process terminates.

R-Swoosh and F-Swoosh [11] are two of the first methods addressing the incremental entity resolution problem. F-Swoosh improves the efficiency of R-Swoosh, but, both algorithms have quadratic cost [11], thus, they cannot scale to large datasets. For instance, executing F-Swoosh (respectively R-Swoosh) on a dataset of 20, 000 records, requires 8 (respectively 9) hours to complete [11].

To improve the efficiency of the incremental entity resolution, we propose two blocking methods, coined INCENTRES and INCENTRES+, that are based on the incremental record linkage algorithm INCSUFFBLOCK discussed in Section 4. Both methods significantly improve the efficiency of the entity resolution process. Specifically, INCENTRES and INCENTRES+ require a few seconds to complete in the same dataset of 20,000 records that F-Swoosh and R-Swoosh required 8 and 9 h to handle. However, since they are based on a blocking heuristic they both might have misses.

Algorithm: INCENTRES

Input: A new record r and the sorted suffixes $SortSuf$ of a dataset D indexed by a StrB-tree SB
Parameters: max_{APP} and min_{LCP}
Local variables: $Candidates$, $similar$, $merged$

```
// Blocking stage of record linkage
1 Candidates = IncSuffBlock(SortSuf, r, max_APP, min_LCP)
// Comparison stage
2 Check r with all records of Candidates and identify the similar records
3 if there are similar records to r in Candidates then
4     Let similar be the most similar
5     Delete similar from D
6     Let merged be the result of merging r and similar
7     IncEntRes(SortSuf, merged, max_APP, min_LCP)
8 else Insert r into D
```

ID	Name	Address
r_1	jim tom	ca usa
r_2	jim tom	california
r_3	jim tom	berkeley ca us

(a) Incoming records

	r_1	r_2	r_3
r_1	—	0.82	0.91
r_2	0.82	—	0.63
r_3	0.91	0.63	—

(b) Similarity scores

Fig. 4. Sample dataset D.

More specifically, INCENTRES takes as input a new record r and the sorted suffixes *SortSuf* of a dataset D indexed by a StrB-tree *SB* (which are required by INCSUFFBLOCK). INCENTRES uses INCSUFFBLOCK to identify the candidate matches (Line 1). These matches are thoroughly checked for similarity (Line 2). If there are many similar records the most similar record *similar* is identified (Line 4) and is removed from D (Line 5). Following, r and *similar* are merged into *merged* (Line 6) and INCENTRES is executed with input the record *merged* (Line 7). If similar records do not exist, then INCENTRES inserts r in the dataset D (Line 8).

Proposition 3. *Algorithm* INCENTRES *can be executed in* $\mathcal{O}(\mu \cdot |r| \cdot (\log |D| + max_{APP}))$ *time, where* μ *is the number of similar records.*

Proof. We assume that similarity and merging functions can be executed in constant time. The time complexity of INCENTRES is determined by the invocations of Algorithm INCSUFFBLOCK (Line 1 and 7). In the worst case scenario, every invocation of INCSUFFBLOCK discovers only one similar record, thus, to discover the μ similar records INCSUFFBLOCK should be invoked μ times. Then, from Proposition 2, the time complexity of Algorithm INCENTRES is $\mathcal{O}(\mu \cdot |r| \cdot (\log |D| + max_{APP}))$.

The following is an example of INCENTRES in operation.

Example 3. *Let us insert the records of* Fig. 4*a using* INCENTRES *into an empty dataset D. Also assume that the similarities between these records are given in* Fig. 4*b, the similarity threshold is 0.8 and the merging function returns the larger input value (for instance merge*("ca usa","california") = "california").

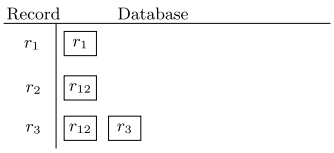
The process is illustrated in Fig. 5*b. Since D is empty, inserting the first tuple* r_1 *is trivial. To insert* r_2, INCENTRES *finds its most similar record* r_1, *removes it from D and inserts the record that results from merging* r_2 *and* r_1. *The merged record, denoted by* r_{12} *is equal to record* r_2. *To insert* r_3, INCENTRES *looks for similar records and since there are none,* r_3 *is inserted into D. The final result is illustrated in* Fig. 5*a.*

The execution of INCENTRES in the above example is unable to take advantage of the strong similarity between r_1 and r_3 (see also Fig. 4b) because when r_1 and r_2 were merged the original records were discarded. Algorithm INCENTRES+ tries to overcome this problem by marking original records as deleted, without actually removing them from the database. This allows future matching with original records that are merged and deleted in previous steps. To keep track of the changes, each original record maintains a link to the merged record as its parent. See for instance Fig. 6b where r_1 and r_2 are merged into r_{12}. All records in the chain of links starting from a specific record (including the record itself) are called *ancestors* of that record. Each merged record will be the root of a tree describing the history of all merge operations that form it.

More specifically, INCENTRES+ starts by inserting the new record r in the dataset D (Line 1). Similar records are identified similarly to INCENTRES (Lines 2 and 3). These matches are thoroughly checked for similarity (Line 3). Only similar records, if any, are kept in set *Candidates* and sorted by non-increasing similarity scores (Line 4). For each similar record *similar* which does not have a common ancestor with r, the following steps are performed. First, the highest common ancestors of r and *similar* are merged into *merged* (Line 8). Second, records r and *similar* maintain links to *merged* (Lines 9, 10). Third, records r and *similar* are marked as deleted (Line 11). Finally, INCENTRES+ is executed with input the record *merged* (Line 12).

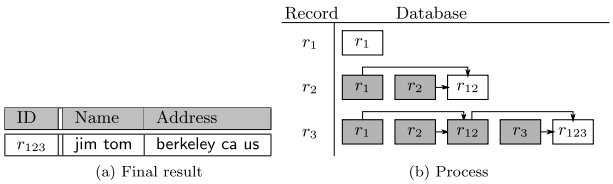
ID	Name	Address
r_{12}	jim tom	california
r_3	jim tom	berkeley ca us

(a) Final result



(b) Process

Fig. 5. Illustration of INCENTRES (Example 3).



(a) Final result (b) Process

Fig. 6. Illustration of INCENTRES+ (Example 4).

Algorithm: INCENTRES+

Input: A new record r and the sorted suffixes *SortSuf* of a dataset D indexed by a StrB-tree SB

Parameters: max_{APP} and min_{LCP}

Local variables: *Candidates*, *similar*, *merged*, h_r, $h_{similar}$

1 Insert r into D
`// Blocking stage of record linkage`
2 *Candidates* = INCSUFFBLOCK(*SortSuf*, r, max_{APP}, min_{LCP})
`// Comparison stage`
3 Check r with all records of *Candidates* and identify the similar records
4 Remove from *Candidates* all records not similar to r and sort the remaining records by non-increasing similarity scores
5 **for** *each record similar* $\in$ *Candidates* **do**
6 **if** *r and similar do not have a common ancestor* **then**
7 Let h_r and $h_{similar}$ be the highest ancestors of r and *similar* respectively
8 Let *merged* be the result of merging h_r and $h_{similar}$
9 Create a link from h_r to its parent *merged*
10 Create a link from $h_{similar}$ to its parent *merged*
11 Mark h_r and $h_{similar}$ as deleted
12 INCENTRES+(*SortSuf*, *merged*, max_{APP}, min_{LCP})

Proposition 4. *Algorithm* INCENTRES+ *can be executed in* $\mathcal{O}(\mu^2 + \mu \cdot |r| \cdot (\log |D| + max_{APP}))$ *time, where* μ *is the number of similar records.*

Proof. Following a similar reasoning to Proposition 3, Algorithm INCENTRES invokes Algorithm INCSUFFBLOCK at most μ times, for each new record, which costs $\mathcal{O}(\mu \cdot |r| \cdot (\log |D| + max_{APP}))$ time. An additional factor arises because for each merge operation, the chain of parent links is followed until reaching the highest ancestor. The maximum chain size is μ. Thus, the time complexity of Algorithm INCENTRES is $\mathcal{O}(\mu^2 + \mu \cdot |r| \cdot (\log |D| + max_{APP}))$.

The following is an example of INCENTRES+ in operation.

Example 4. *Let us repeat* Example 3 *using* INCENTRES+ *this time. The process is illustrated in* Fig. 6*b. Since D is empty, inserting the first tuple* r_1 *is trivial. To process* r_2, INCENTRES+ inserts r_2 in D and computes *Candidates* = $\{r_1\}$ *(Lines 2–4). Since* r_1 *and* r_2 *do not have parents, they are marked as deleted and the record* r_{12} *that results in from merging* r_2 *and* r_1 *is inserted to D. Records* r_1 and r_2 *keep links to the merged record, indicating parent relationship. The merged record* r_{12} *is equal to record* r_2*. To process* r_3, INCENTRES+ *inserts* r_3 *in D* and computes *Candidates* = $\{r_1\}$ *(Lines 2–4). Note that* r_1 *appears in Candidates although it is marked as deleted. Record* r_3 *is merged with the highest ancestor of* r_1 *which is* r_{12}*, resulting in the merged record* r_{123}*. Both* r_3 *and* r_{12} *are marked as deleted. Records* r_3 *and* r_{12} *keep links to their parent record* r_{123}*. The final result is illustrated in* Fig. 6*a.*

Algorithms INCENTRES and INCENTRES+ can also be implemented to incorporate external storage. Similarly to Algorithm INCSUFFBLOCK in Section 5, the StrB-tree *SB* is divided horizontally into two parts; the top that contains the root is stored in memory and the bottom that contains the leaves is stored in disk. Moreover, for Algorithm INCENTRES+ the additional structure that maintains the history of deletions and merge operations may also be stored in disk.

6. Experiments

We start by discussing our experimental setting (Section 6.1). Then, we present the evaluation of our methods developed to handle record linkage (Section 6.2), incremental record linkage (Section 6.3) and incremental entity resolution (Section 6.4). Finally, in Section 6.5 we evaluate the efficiency of disk-based implementation.

Table 1
Properties of real-world datasets.

Dataset	Num of records	Original	Duplicates	Duplicates/record
MOVIES	50, 797	56%	44%	up to 1
PATIENTS	100, 000	84%	16%	up to 8
DBPEDIA	3, 354, 773	73%	27%	up to 1

6.1. Experimental setting

To evaluate the proposed methods, we have used real-world and synthetic datasets. The real-world datasets used are: (a) the MOVIES dataset [2], which consists of a collection of 50, 797 movie records from IMDB and DBPedia. (b) the dataset of [6] that contains comparison results between 5, 749, 132 record pairs. The original records are not publicly available, since they represent critical and private information about patients, but, they can partially be reconstructed by the comparison results. The reconstructed dataset, coined PATIENTS, contains 100, 000 records. (c) the DBPEDIA dataset [2], which consists of a collection of 3, 354, 773 records from two different versions of the DBPedia Infobox dataset.

Note that both MOVIES and DBPEDIA datasets consist originally of two duplicate-free subsets from heterogeneous sources. To be compatible with our framework, we concatenated the two duplicate-free subsets for both datasets. Detailed properties of all real-world datasets are illustrated in Table 1.

Similarly to previous record linkage research efforts [3,19,23,27], to create synthetic datasets, we have used the Febrl tool [27]. Febrl creates datasets with records having 15 words on average. Similarly to [23], we have created two datasets called CLEAN and DIRTY each containing 10^6 records. These datasets differ on the number of original and duplicate records, the number of duplicates per record, and the number of modifications per attribute/record. Their detailed properties are illustrated in Table 2.

All evaluated methods were implemented in C++ and all experiments were executed on a machine with two Intel 6-core CPUs at 2.67 GHz and 16 GB RAM running Linux. Each time efficiency experiment was run 5 times and the average running time is reported. System cache is cleared before running each experiment.

6.2. Record linkage

We compare our record linkage method SUFFBLOCK (Section 3) with SAB [18] and the work of [19] which we call RSAB. RSAB builds an inverted index to store all suffixes. To generate candidate matches for a given record, all matching suffixes are retrieved by querying the inverted index. More details appear in Ref. [19]. To this end, we have used the CLEAN and DIRTY datasets. For the SUFFBLOCK and SAB algorithms, we have set $max_{APP} = 20$ and $min_{LCP} = 1$.

We focus on and illustrate the running efficiency and ignore the loading, the preprocessing and the output of data which are the same for all methods. Our results are illustrated in Fig. 7 and Table 3. Clearly, SUFFBLOCK is at least 5 times faster than RSAB and an order of magnitude faster than SAB. For example, for the DIRTY dataset that contains 500, 000 records, SUFFBLOCK, RSAB and SAB require 1.62, 7.44 and 17.87 s, respectively. Note that the construction of array *SortSuf* requires 1.68 s, thus, the total execution time of the above blocking methods is 3.30, 9.12 and 19.55, respectively. For the DIRTY dataset that contains 1, 000, 000 records, SUFFBLOCK, RSAB and SAB require 3.36, 18.08 and 43.52 s, respectively. In this case the construction of array *SortSuf* requires 3.90 s. Algorithm SUFFBLOCK outperforms RSAB because RSAB requires several queries to the inverted index that stores all suffixes, which suffer from poor locality. Also SUFFBLOCK is faster than SAB because it uses a dynamic sliding window which is very cache efficient. On the contrary, SAB conducts binary searches on the array *SortSuf* to locate blocks of similar suffixes, which suffer from poor locality. Also, SUFFBLOCK avoids iterating over all the prefixes of a certain suffix as SAB does. For more details refer to Section 3 and [18,19].

To evaluate the accuracy of the blocking stage (discussed in Section 2) offered by SUFFBLOCK, SAB and RSAB, we use pairs completeness [23,28,29] and pairs quality [23,29] metrics. Given a dataset D, let L be the set of actual linked record pairs in D and let R be the set of record pairs that result in by the blocking stage. We, then, have [23,29]:

- *Pairs completeness*, corresponds to recall, defined as $\frac{|R \cap L|}{|L|}$, shows the fraction of the correct pairs that are outputted by the blocking stage.
- *Pairs quality*, corresponds to precision, defined as $\frac{|R \cap L|}{|R|}$, shows the fraction of the correct pairs in the result R of the blocking stage.

SUFFBLOCK and SAB achieve the same accuracy since they produce exactly the same result. The output of RSAB is different but achieves a similar accuracy in all our experiments. Table 4 shows the similarity of accuracy results of running SUFFBLOCK, SAB and RSAB on the DIRTY and DBPEDIA datasets. Note that pairs quality values are relatively low in all experiments, since the majority of pairs

Table 2
Properties of synthetic datasets.

Dataset	Num of records	Original	Duplicates	Duplicates/record	Modifications/attribute	Modifications/record
CLEAN	10^6	80%	20%	up to 3	up to 1	up to 3
DIRTY	10^6	60%	40%	up to 9	up to 3	up to 10

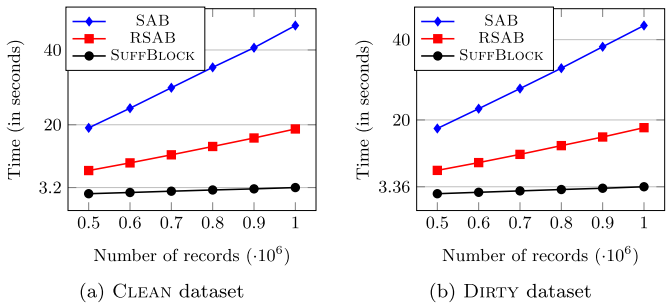


Fig. 7. Time efficiency comparison of record linkage for synthetic datasets.

Table 3
Time efficiency comparison of record linkage for real-world datasets (Times are shown in seconds).

Algorithm	MOVIES	PATIENTS	DBPEDIA
SortSuf construction[a]	0.39	0.19	50.65
SUFFBLOCK	0.37	0.09	40.44
RSAB	2.12	0.41	294.84
SAB	4.93	1.24	778.04

[a] *SortSuf* construction is an initial step common to all algorithms.

Table 4
Record linkage (SUFFBLOCK and SAB vs. RSAB).

Dataset	Algorithm	Pairs Completeness (%)	Pairs Quality (%)
DIRTY	SUFFBLOCK/SAB	98.83	7.44
	RSAB	98.43	9.46
DBPEDIA	SUFFBLOCK/SAB	95.89	0.52
	RSAB	93.08	0.73

outputted by the blocking stage are not correct and will be filtered by the comparison stage. More details exist in Section 6.4.

6.3. Incremental record linkage

In this section, we experimentally evaluate our incremental record linkage method INCSUFFBLOCK. As discussed in Section 4, INCSUFFBLOCK provides record linkage in an incremental and blocking-based fashion and can be tuned by the min_{LCP} and max_{APP} parameters. In a nutshell, decreasing min_{LCP} and/or increasing max_{APP} renders INCSUFFBLOCK to consider more record pairs which decreases pairs quality and increases pairs completeness; increasing min_{LCP} and/or decreasing max_{APP} has the opposite effect.

Similarly to Section 6.2, to compare INCSUFFBLOCK we may devise and use incremental variations of the SAB and RSAB algorithms. Unfortunately, such variations do not exist in the related literature. Also, by studying the properties and data structures of the SAB or RSAB algorithms, there does not seem to be a straightforward way to construct an efficient incremental counterpart. Additionally, to the best of our knowledge, there does not exist another algorithm with the incremental and blocking characteristics of INCSUFFBLOCK. Thus, we base our comparison on Multi-Pass Blocking [13], a method commonly used in practice [14,22]. We appropriately modify the original method of [13] to support incremental record linkage while producing the same output. We refer to this method as MPB. As in the original method [13], MPB is tuned by the size of the working window. Increasing the size of the window results in more record pairs which decreases pairs quality and increases pairs completeness; decreasing the window size has the opposite effect.

To evaluate the accuracy of the blocking stage (discussed in Section 2) offered by INCSUFFBLOCK and MPB, we use the pairs completeness metric defined in Section 6.2 and the *number of pairs for comparison* metric which counts the total number of pairs outputted by the blocking stage and fed to the comparison stage. The latter metric is analogous to the pairs quality metric defined in Section 6.2. We compare INCSUFFBLOCK and MPB using all datasets of Section 6.1. We configure INCSUFFBLOCK by setting min_{LCP} to 1 and tuning max_{APP}

Table 5
Parameter tuning of INCSUFFBLOCK and MPB.

Parameter	CLEAN	DIRTY	MOVIES	PATIENTS	DBPEDIA
INCSUFFBLOCK max_{APP}	17–50	17–500	25–200	5–16	11–32
MPB working window size	2–6	2–100	6–64	2–6	2–7

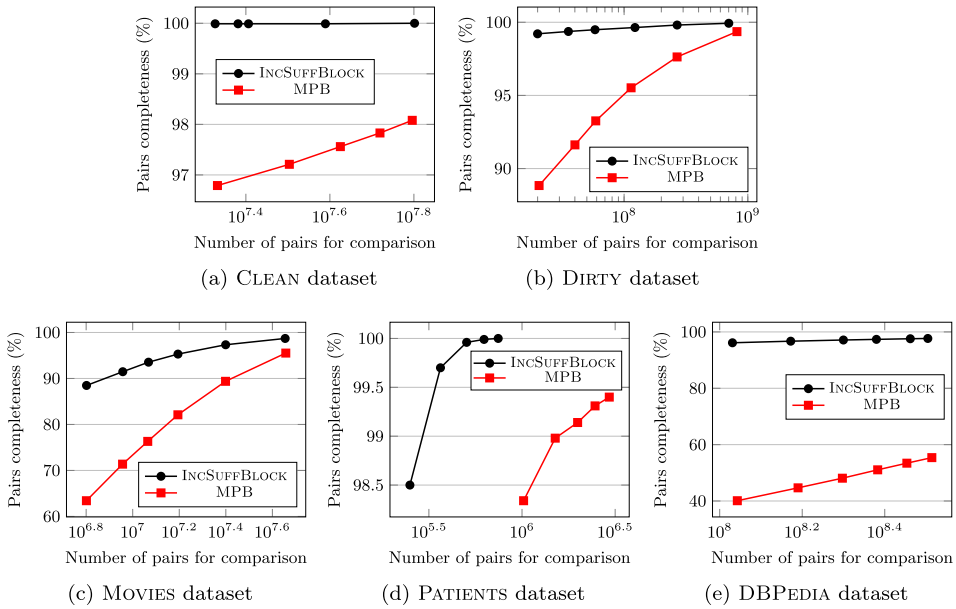


Fig. 8. Incremental record linkage (incsuffblock vs. mpb).

as shown in Table 5. We configure MPB by setting the blocking keys to the set of all record words and tuning the size of the working window as shown in Table 5. Our results are illustrated in Fig. 8. Clearly incsuffblock outperforms mpb for all these datasets. Specifically, for the same number of pairs for comparison, incsuffblock achieves better pairs completeness than mpb by up to 3%, 10%, 30% and 50% for the clean, dirty, movies and dbpedia datasets respectively. For the patients dataset, the minimum number of pairs for comparison for mpb is about 10^6 (the leftmost red square in Fig. 8d); for these comparisons mpb has 98.34% pairs completeness. Contrary, incsuffblock achieves 98.50% pairs completeness with just 250, 000 pairs for comparison. It also worth to note that with about 750, 000 pairs for comparison incsuffblock achieves 100% pairs completeness (the rightmost black circle in Fig. 8d).

To evaluate the efficiency of incremental record linkage, we perform the following two experiments.

- In the first one, we start from an empty database and insert one record at a time until all records are inserted. In Fig. 9a, c and 9e, we report the total insertion time at specific points of time for the clean, dirty and dbpedia datasets respectively.
- In the second experiment, we start from a nonempty database and insert 100 records, one record at a time, and we report their average insertion time. Our results are illustrated in Fig. 9b, d and 9f for the clean, dirty and dbpedia datasets respectively.

For both experiments, we compare incsuffblock with mpb; their efficiency for the clean, dirty and dbpedia datasets is illustrated in Fig. 9. Note that MPB and incsuffblock do not produce the same output, thus, MPB is tuned to produce a close pairs completeness to incsuffblock. Our results show that incsuffblock outperforms MPB by at least 8 times for the clean dataset, 2 times for the dirty dataset, and 14 times for the dbpedia dataset. incsuffblock improved efficiency can be attributed to its mechanism that identifies the important common sequences (that do not appear a lot in the dataset) and deserve further processing. Contrary, MPB lacks such a mechanism and generates and processes *strictly more* pairs.

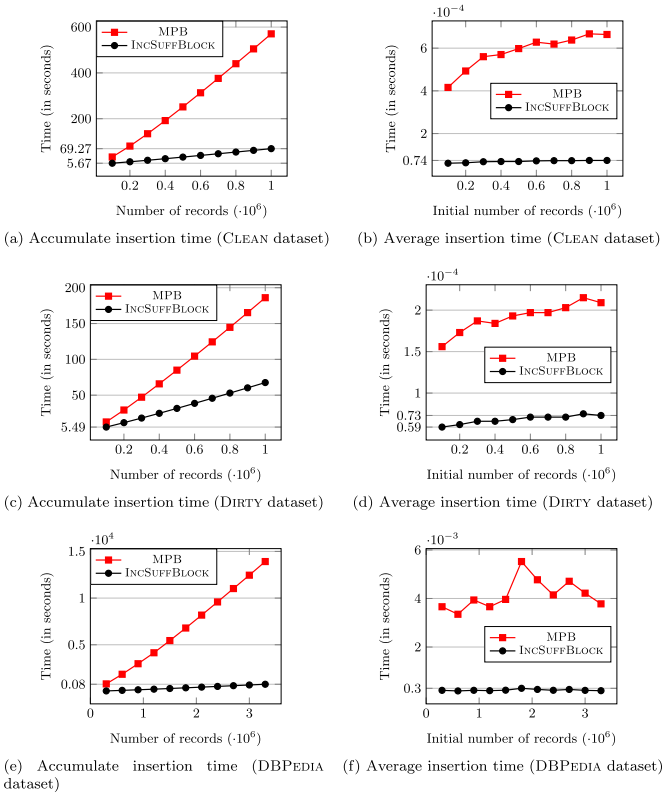
6.4. Incremental entity resolution

In this section, we experimentally evaluate our incremental entity resolution methods incentres and incentres+. As discussed in Section 5, incentres and incentres+ provide incremental and blocking-based entity resolution that can be tuned by the min_{LCP} and max_{APP} parameters. Overall, decreasing min_{LCP} and/or increasing max_{APP} renders incentres and incentres+ to consider more record pairs which decreases pairs quality and increases pairs completeness; increasing min_{LCP} and/or decreasing max_{APP} has the opposite effect.

In the related literature, there does not exist another algorithm with the incremental and blocking characteristics of incentres and incentres+. Thus, we have chosen to compare with R-Swoosh [11] which is a state-of-the-art incremental entity resolution method.

Metrics to evaluate incremental entity resolution algorithms, typically include pair precision [4,8,13], pair recall [4,8,13], pair F_1 [8], and merge distance [8]. Given a dataset D, let L be the set of actual linked record pairs in D and let R be the set of record pairs that result in by the algorithm under evaluation. We, then, have [8]:

- *Pair precision*, defined as $\frac{|R \cap L|}{|R|}$, shows the fraction of the correct pairs in the result R of the evaluated algorithm.
- *Pair recall*, defined as $\frac{|R \cap L|}{|L|}$, shows the fraction of the correct pairs that are discovered by the evaluated algorithm.

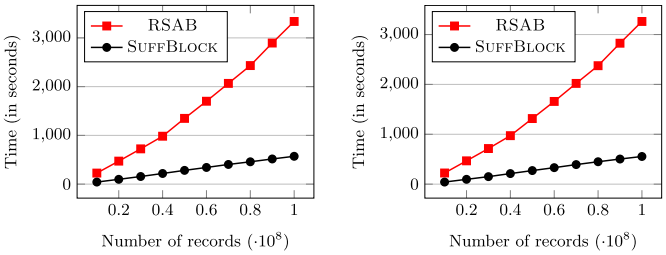


(a) Accumulate insertion time (Clean dataset) (b) Average insertion time (Clean dataset)

(c) Accumulate insertion time (Dirty dataset) (d) Average insertion time (Dirty dataset)

(e) Accumulate insertion time (DBPedia dataset) (f) Average insertion time (DBPedia dataset)

Fig. 9. Time efficiency comparison of incremental record linkage.



(a) Clean$_{\times 100}$ dataset (b) Dirty$_{\times 100}$ dataset

Fig. 10. Time efficiency comparison of on-disk record linkage (suffblock vs. rsab).

High pair precision values indicate that the evaluated algorithm discovers a small fraction of incorrect pairs. High pair recall values indicate that the evaluated algorithm discovers a large fraction of the actual pairs. Pair F_1 is defined as the harmonic mean of pair precision and pair recall, thus, it combines these metrics. Merge distance metric undertakes a different approach. Instead of evaluating pairs, it merges pairs having a common element into clusters and evaluates these clusters. Specifically, merge distance is defined as the number of splits and merges required to transform the clusters generated by the evaluated algorithm into the clusters that result in by the actual linked record pairs of the dataset. Lower values of the metric denote better accuracy. Formal definitions of the above metrics appear in Ref. [8].

In order to evaluate Algorithms R-Swoosh, incentres and incentres+, we must find a similarity measure that accurately differentiates between duplicates and non-duplicates during the comparison stage (Line 2 in Algorithm incentres and Line 3 in Algorithm incentres+). The comparison stage thoroughly checks candidate records produced by the blocking stage for similarity with the input record which filters out most candidate pairs.

We use the Jaro similarity [30] as the similarity measure for the clean and dirty datasets. We set the Jaro similarity threshold to 0.80 and 0.75 for the clean and dirty datasets respectively. Unfortunately, we did not find an accurate similarity measure suitable to the movies and dbpedia datasets, hence they are not used for evaluation.[1] Also, the patients dataset cannot be used for evaluation since

[1] We tried to use the Jaccard similarity which was proposed by Ref. [29] for these datasets, however, it has very low accuracy and cannot be used

Table 6
Comparison for incremental entity resolution methods (best values appear in red).

Dataset	Algorithm	*Pair Prec.* (%)	*Pair Recall* (%)	*Pair* F_1 (%)	*Merge Dist.*	Time (minutes)
		Higher is better			Lower is better	
CLEAN	R-Swoosh	99.974	99.621	99.798	559	11,124.55
	INCENTRES	99.986	99.105	99.543	1208	1.92
	INCENTRES +	99.980	99.820	99.900	303	2.05
DIRTY	R-Swoosh	98.185	96.079	97.121	15,935	12,525.14
	INCENTRES	99.589	89.886	94.489	22,304	1.97
	INCENTRES +	98.829	98.566	98.697	6157	2.57

to apply the comparison stage and the merging functions (Line 6 of INCENTRES and Line 8 of INCENTRES +) the actual data are required (we remind the reader that the PATIENTS dataset contains comparison results and not the actual data; see also Section 6.1). The results of the comparison are illustrated in Table 6.

For the accuracy metrics, note that INCENTRES + performs more comparisons than INCENTRES since it compares every new record with the records of the dataset *and the stored deleted* records (while INCENTRES compares only with the records of the dataset). These additional comparisons render INCENTRES + to discover more candidate pairs than INCENTRES. A small portion of these additional pairs are incorrect, which may cause INCENTRES + to have slightly lower pair precision (by 0.006% and 0.76% for the CLEAN and DIRTY datasets respectively). However, the larger portion of these additional discovered pairs are correct, which increases the pair recall of INCENTRES + (by 0.71% and 8.68% for the CLEAN and DIRTY datasets respectively). Moreover, since the number of additional correct pairs significantly exceeds the number of additional incorrect pairs, INCENTRES + has better pair F_1 (by 0.36% and 4.21% for the CLEAN and DIRTY datasets respectively) and merge distance (by a factor of 4) than INCENTRES.

For time efficiency, as expected Algorithms INCENTRES and INCENTRES + , due to their proposed blocking, outperform R-Swoosh by 3.5 orders of magnitude for both the CLEAN and DIRTY datasets.

6.5. Disk-based implementation

All previous experiments (Sections 6.2–6.4) were conducted exclusively in-memory (that is, all datasets and indexes are stored in main memory). In this section, we will consider the variations of the introduced algorithms that also facilitate disk storage. These variations are obviously slower than the memory-based counterparts, but, allow us to perform the record linkage and entity resolution tasks to datasets that cannot fit in main memory.

We will start with the variation of the SUFFBLOCK algorithm that uses disk storage (see Section 6.2 for more details). To experimentally evaluate the efficiency of SUFFBLOCK, we need datasets that require disk storage. To this end, we use $\text{CLEAN}_{\times 100}$ and $\text{DIRTY}_{\times 100}$ datasets, which have similar properties to the CLEAN and DIRTY datasets but each contains 100 million records; a total number or records 100 times larger than the CLEAN and DIRTY datasets.

Similarly to Section 6.2, we compare the disk-based variations of algorithms SUFFBLOCK and RSAB. Our results are illustrated in Fig. 10. Similarly to the main memory case (illustrated in Fig. 7), SUFFBLOCK is at least 5 times faster than RSAB. The disk-based variation of SUFFBLOCK is faster than the respective variation of RSAB because SUFFBLOCK performs the whole blocking process by one disk seek followed by sequential disk access. Contrary, RSAB conducts several queries to an inverted index that stores the suffixes, which suffer from poor locality.

The behavior of the external memory implementations of Algorithms INCSUFFBLOCK, INCENTRES and INCENTRES + are similar and are omitted for brevity.

7. Related work

This section presents the most important record linkage and entity resolution algorithms. Motivated by real applications such as linking vital records [5] and medical records [6], early record linkage and entity resolution techniques were non-blocking. Such techniques simply compare all existing record pairs using a domain-dependent similarity function to identify similar records, then, if required, they merge the similar records using a domain-dependent procedure [11]. Based on pairwise record comparisons, records can optionally be clustered using probabilistic techniques [3,31], machine learning techniques such as support vector machines [18], or graph clustering techniques [10].

Blocking-based techniques are required for large datasets. The importance of blocking is discussed in Section 2. Suffix-based blocking [18,19] utilizes record suffixes to group records sharing common sequences. Section 2 explains the differences between existing suffix-based blocking approaches, including the approach introduced in this paper. Suffix-based blocking is efficient, accurate and does not require any domain specific knowledge. Also, it can be tuned by only two easy-to-set and intuitive parameters.

(*footnote continued*)
effectively for entity resolution. The Jaccard similarity is used in Ref. [29] only for reporting time efficiency results, but its effect on entity resolution accuracy is not addressed.

There exist various blocking methods in the literature. In Multi-pass blocking [13,14], records are sorted according to various blocking keys extracted from record fields. Records occurring in the same window associated with any blocking key are candidates for comparison. This approach is similar to suffix-based blocking in utilizing a moving window over sorted records, and in being computationally efficient. However, its accuracy depends on the selection of blocking keys. Q-gram based indexing [9,15,16] utilizes approximate string matching. Records sharing a minimum number of common q-grams are candidates for comparison. Contrary to suffix-based blocking, this technique is computationally expensive and applicable only to small datasets [23]. Canopy clustering [17], selects a random record and compares it with all other records using a distance measure, such as the tf-idf measure. This process is repeated several times, generating canopies which may overlap. Records in the same canopy are candidates for comparison. The main difficulty of canopy clustering is to determine the values of parameters, the distance threshold and the number of canopies, without falling into under-selection or over-selection of candidate record pairs. In contrast, suffix-based blocking can be tuned by only two easy-to-set and intuitive parameters. Mapping [4] techniques map blocking key values of each record to points in a multidimensional space. Two records are candidates for comparison if their mapping points are within a chosen distance threshold. The process can be repeated using different blocking keys. The selection of the blocking keys and parameter values are determined by experiments on a sample of records [4]. Contrary to suffix-based blocking, this technique is computationally expensive unless blocking keys are small in size. MFIBlocks [20] iteratively applies an algorithm for mining maximal frequent itemsets in order to generate record blocks. Similarly to suffix-based blocking, MFIBlocks attempts to avoid the manual selection of blocking keys, but it is relatively computationally expensive. Meta-blocking [2,21,32,33] is a general blocking framework for highly heterogeneous databases. Meta-blocking differs from traditional blocking in incorporating record comparisons during the blocking stage, in order to adjust record blocks. Meta-blocking can utilize an underlying traditional blocking method, such as suffix-based blocking. Progressive duplicate detection [34,35] techniques build upon traditional blocking in order to find most duplicates early while execution. The user can terminate the execution whenever the reported duplicates are satisfying, or wait until all duplicates are reported.

Incremental blocking-based techniques are required to solve the incremental problem efficiently on large datasets. We introduced an incremental suffix-based blocking algorithm in Section 4. Multi-pass blocking [13,14] can be efficiently extended to be incremental [22]. Q-gram based indexing [9,15,16] can be extended to be incremental as well, however, it is computationally very expensive when applied to large datasets [23]. Extending the remaining blocking methods to be incremental is difficult and not straightforward. Table 7 provides a comparison for traditional blocking techniques.

There exist incremental techniques in the literature that address different aspects of the problem, which are not the focus of this paper. The work of [3] focuses on minimizing communication overhead, given that the dataset exists on remote site where procedures are not allowed to run, and training data is available to build a probabilistic model. The work of [36] focuses on efficiently handling the change of entity resolution rules and data schema to reproduce entity resolution results based on previously computed results. The work of [10] separates the record linkage process into three independent steps: (a) the blocking phase, (b) the similarity computation phase and (c) the graph clustering phase. These phases are independent and orthogonal. This means that each of these phases can be considered and optimized separately. The work of [10] focuses on the third graph clustering phase and proposes a technique to incrementally maintain disjoint clusters of records, given similarity scores between record pairs. More specifically, records assumed to belong to the same entity are maintained in the same cluster. These clusters are maintained and updated incrementally after the insertion of new records.

8. Conclusions and discussion

The problems of record linkage, entity resolution and their incremental counterparts arise in the context of several important applications. A related research area is how to design a similarity function that detects whether two records represent the same entity. Such similarity function is clearly domain-dependent. Another research area arises with large datasets, which deals with avoiding the quadratic cost comparisons of all record pairs using the specified similarity function. Blocking methods are concerned with minimizing the number of record comparisons without excluding valid linked records. Since the purpose of blocking is to avoid the quadratic cost comparisons, time efficiency is an essential criteria of blocking methods. In all blocking techniques, modifying the associated parameters shows a trade-off between excluding more comparisons and including more valid linked records. Suffix blocking is one of the most effective blocking methods, although it may not be the best choice for some situations, such as heterogeneous databases. Some other blocking methods might be more accurate that suffix blocking. Such methods suffer from one or more of these drawbacks: (a) Relatively computationally expensive. (b) Require domain specific knowledge. (c) Can not be adjusted to be incremental. (d) Require manual setting of many parameters.

Table 7
Comparison for traditional blocking techniques.

Blocking technique	Accuracy	Time efficiency	Incremental	Parameters
Suffix blocking [18,19]	High	High	Yes	Few
Multi-pass blocking [13,14]	Moderate	High	Yes	Few
Q-gram based indexing [9,15,16]	Moderate	Low	Yes	Few
Canopy clustering [17]	High	High	No	Many
Mapping [4]	High	Moderate	No	Many
MFIBlocks [20]	High	Moderate	No	Few

In this paper, we have considered the non-incremental variation of record linkage and presented a method that is faster and achieves similar accuracy to the current state-of-the-art suffix-based blocking method. Following, we have considered the incremental variation of record linkage and proposed a novel incremental suffix-based blocking mechanism that outperforms existing incremental methods in terms of blocking accuracy and efficiency. Also, we have considered incremental entity resolution and presented two novel methods based on suffix blocking which are more efficient than existing methods. We have implemented and extensively experimentally evaluated all our methods. We have proposed a memory-based implementation and a disk-based implementation which is able to handle very large datasets.

Funding

This work was supported by King Abdullah University of Science and Technology (KAUST), Thuwal, Saudi Arabia.

References

[1] N. Polyzotis, S. Skiadopoulos, P. Vassiliadis, A. Simitsis, N. Frantzell, Meshing streaming updates with persistent data in an active data warehouse, Trans. Knowl. Data Eng. 20 (7) (2008) 976–991.
[2] G. Papadakis, E. Ioannou, T. Palpanas, C. Niederee, W. Nejdl, A blocking framework for entity resolution in highly heterogeneous information spaces, Trans. Knowl. Data Eng. 25 (12) (2013) 2665–2682.
[3] D. Dey, V.S. Mookerjee, D. Liu, Efficient techniques for online record linkage, Trans. Knowl. Data Eng. 23 (3) (2011) 373–387.
[4] L. Jin, C. Li, S. Mehrotra, Efficient record linkage in large data sets, DASFAA, 2003, pp. 137–146.
[5] H.B. Newcombe, J.M. Kennedy, S.J. Axford, A.P. James, Automatic linkage of vital records, Science 130 (3381) (1959) 954–959.
[6] M. Sariyar, A. Borg, K. Pommerening, Controlling false match rates in record linkage using extreme value theory, J. Biomed. Inf. 44 (2011) 648–654.
[7] H.L. Dunn, Record linkage, Am. J. Pub. Health Nations Health 36 (12) (1946) 1412–1416.
[8] D. Menestrina, S. Whang, H. Garcia-Molina, Evaluating entity resolution results, PVLDB 3 (1) (2010) 208–219.
[9] M. Hadjieleftheriou, N. Koudas, D. Srivastava, Incremental maintenance of length normalized indexes for approximate string matching, SIGMOD, 2009, pp. 429–440.
[10] A. Gruenheid, X.L. Dong, D. Srivastava, Incremental record linkage, PVLDB 7 (9) (2014) 697–708.
[11] O. Benjelloun, H. Garcia-Molina, D. Menestrina, Q. Su, S.E. Whang, J. Widom, Swoosh: a generic approach to entity resolution, VLDB J. 18 (1) (2009) 255–276.
[12] M. Bilenko, B. Kamath, R.J. Mooney, Adaptive blocking: learning to scale up record linkage, ICDM, 2006, pp. 87–96.
[13] M.A. Hernández, S.J. Stolfo, The merge/purge problem for large databases, SIGMOD, 1995, pp. 127–138.
[14] L. Kolb, A. Thor, E. Rahm, Multi-pass sorted neighborhood blocking with MapReduce, Comput. Sci. Res. Dev. 27 (1) (2012) 45–63.
[15] S. Chaudhuri, K. Ganjam, V. Ganti, R. Motwani, Robust and efficient fuzzy match for online data cleaning, SIGMOD, 2003, pp. 313–324.
[16] L. Gravano, P.G. Ipeirotis, H.V. Jagadish, N. Koudas, S. Muthukrishnan, D. Srivastava, Approximate string joins in a database (almost) for free, VLDB, 2001, pp. 491–500.
[17] A. McCallum, K. Nigam, L.H. Ungar, Efficient clustering of high-dimensional data sets with application to reference matching, KDD, 2000, pp. 169–178.
[18] A.N. Aizawa, K. Oyama, A fast linkage detection scheme for multi-source information integration, WIRI, 2005, pp. 30–39.
[19] T. de Vries, H. Ke, S. Chawla, P. Christen, Robust record linkage blocking using suffix arrays and bloom filters, Trans. Knowl. Discov. Data 5 (2011) 9:1–9:27.
[20] B. Kenig, A. Gal, MFIBlocks: an effective blocking algorithm for entity resolution, Inf. Syst. 38 (6) (2013) 908–926.
[21] G. Papadakis, G. Koutrika, T. Palpanas, W. Nejdl, Meta-blocking: taking entity resolution to the next level, Trans. Knowl. Data Eng. 26 (8) (2014) 1946–1960.
[22] M.J. Welch, A. Sane, C. Drome, Fast and accurate incremental entity resolution relative to an entity knowledge base, CIKM, 2012, pp. 2667–2670.
[23] P. Christen, A survey of indexing techniques for scalable record linkage and deduplication, Trans. Knowl. Data Eng. 24 (2012) 1537–1555.
[24] Ferragina Crauser, A theoretical and experimental study on the construction of suffix arrays in external memory, Algorithmica 32 (1) (2002) 1–35.
[25] P. Ferragina, R. Grossi, The string b-tree: a new data structure for string search in external memory and its applications, J. ACM 46 (2) (1999) 236–280.
[26] J.C. Na, K. Park, Simple implementation of string b-trees, SPIRE, 2004, pp. 214–215.
[27] P. Christen, Febrl - an open source data cleaning, deduplication and record linkage system with a graphical user interface, KDD, 2008, pp. 1065–1068.
[28] M.G. Elfeky, V.S. Verykios, A.K. Elmagarmid, TAILOR: a record linkage toolbox, ICDE, 2002, pp. 17–28.
[29] G. Papadakis, J. Svirsky, A. Gal, T. Palpanas, Comparative analysis of approximate blocking techniques for entity resolution, PVLDB 9 (9) (2016) 684–695.
[30] M.A. Jaro, Advances in record-linkage methodology as applied to matching the 1985 census of tampa - Florida, J. Am. Stat. Assoc. 84 (406) (1989) 414–420.
[31] M.H. Hof, A.C. Ravelli, A.H. Zwinderman, A probabilistic record linkage model for survival data, J. Am. Stat. Assoc. 112 (520) (2017) 1504–1515.
[32] G. Papadakis, G. Papastefanatos, T. Palpanas, M. Koubarakis, Scaling entity resolution to large, heterogeneous data with enhanced meta-blocking, EDBT, 2016, pp. 221–232.
[33] G. Simonini, S. Bergamaschi, H. Jagadish, Blast: a loosely schema-aware meta-blocking approach for entity resolution, PVLDB 9 (12) (2016) 1173–1184.
[34] T. Papenbrock, A. Heise, F. Naumann, Progressive duplicate detection, Trans. Knowl. Data Eng. 27 (5) (2015) 1316–1329.
[35] D. Firmani, B. Saha, D. Srivastava, Online entity resolution using an oracle, PVLDB 9 (5) (2016) 384–395.
[36] S.E. Whang, H. Garcia-Molina, Incremental entity resolution on rules and data, VLDB J. 23 (1) (2014) 77–102.